Programación II

Profesorado de Informática

CERP del Suroeste

Resumen del capítulo 5 del libro Ingeniería de Software "Comprensión de los requerimientos" de Roger Pressman (7ma edición).

Docente

Prof. Domingo Pérez González

Estudiante

Camila Fleitas Pérez

1.3 Metodología para desarrollo de Sistemas OO:

2. Crear un documento en grupo resumen, con imágenes, del capítulo 5 del libro Ingeniería de Software de Roger Pressman (7ma edición). “Comprensión de los requerimientos”.

Ingeniería de requerimientos: La ingeniería de requerimientos es una disciplina fundamental que inicia en la fase de comunicación y se extiende a lo largo del modelado, adaptándose continuamente al proceso, al proyecto, al producto y al equipo de trabajo. Actúa como un puente entre la concepción y la construcción del software, proporcionando una perspectiva integral que permite:

- Comprender el contexto global y las necesidades específicas que deben resolver el diseño y la implementación.
- Establecer prioridades claras para guiar el orden de desarrollo.
- Analizar la información, funciones y comportamientos que darán forma al diseño final.

Ofrece el marco metodológico para entender al cliente, evaluar la factibilidad, negociar soluciones viables, especificar sin ambigüedades, validar el modelo y administrar la evolución de los requerimientos a lo largo de todo el ciclo de vida del desarrollo. La ingeniería de requerimientos comprende siete actividades interrelacionadas que suelen ejecutarse de forma iterativa y en paralelo según las necesidades del proyecto:

1. **Concepción:** Inicia el proyecto a partir de una necesidad de negocio o un nuevo mercado. Los participantes del negocio definen el caso de negocio, evalúan la factibilidad general y delimitan un alcance funcional preliminar para iniciar el diálogo con el equipo de ingeniería.
2. **Indagación:** Recaba los objetivos y el contexto de uso mediante la comunicación con los clientes e interesados. Aborda los problemas clásicos de la recolección:
    - *Alcance:* Fronteras del sistema mal definidas o detalles técnicos innecesarios.
    - *Entendimiento:* Ambigüedades, omisiones por considerarlas "obvias", requerimientos en conflicto o no verificables.
    - *Volatilidad:* Cambios constantes en las necesidades a lo largo del tiempo.
3. **Elaboración:** Refina la información obtenida para construir el modelo de análisis. Se centra en el desarrollo de escenarios de usuario (casos de uso), la definición de clases de análisis (atributos y servicios), la identificación de relaciones y la generación de diagramas UML.
4. **Negociación:** Modera las peticiones en conflicto y la limitación de recursos. Los participantes priorizan características, evalúan costos y riesgos, y modifican o combinan requerimientos hasta alcanzar un acuerdo de beneficio mutuo (*ganar-ganar*).
5. **Especificación:** Formaliza el comportamiento y restricciones del sistema. Puede adoptar diversos formatos (documentos narrativos, modelos gráficos, prototipos o modelos matemáticos), adaptando la formalidad al tamaño y complejidad del proyecto.
6. **Validación:** Examina la especificación para garantizar que los requerimientos sean claros, consistentes, completos y no ambiguos. Se apoya en revisiones técnicas formales con clientes e ingenieros para detectar errores antes de la fase de diseño.

7. **Administración:** Controla y rastrea los cambios en los requerimientos a medida que el sistema evoluciona. Aplica mecanismos de trazabilidad y gestión de configuración durante todo el ciclo de vida del software.

Establecer las Bases: Aunque lo ideal es que clientes e ingenieros de software trabajen juntos en el mismo equipo, en la práctica suelen surgir barreras como la distancia geográfica, ideas vagas sobre el sistema, conflictos de opinión, poco conocimiento técnico o disponibilidad limitada de tiempo. Para superar estas restricciones reales, se deben definir etapas estructuradas que inicien el proyecto con un entendimiento claro y alineado de los requerimientos.

5.2.1 Identificación de los Participantes (*Stakeholders*): Un participante es cualquier persona o grupo que se beneficie directa o indirectamente del sistema en desarrollo (gerentes, clientes, usuarios finales, ingenieros de soporte, entre otros). Dado que cada uno tiene diferentes expectativas, beneficios y riesgos, durante la concepción se debe confeccionar una lista inicial de contactos que se irá ampliando progresivamente preguntando a cada integrante a quién más se debería consultar.

5.2.2 Reconocer los Múltiples Puntos de Vista: Al existir diversos grupos de participantes, los requerimientos se analizan desde distintas perspectivas:

- **Mercadotecnia:** Busca funciones atractivas para facilitar las ventas.
- **Gerentes del negocio:** Priorizan cumplir el presupuesto y los plazos de entrega.
- **Usuarios finales:** Prefieren características familiares y fáciles de usar.
- **Ingenieros de software y soporte:** Se enfoca en la infraestructura técnica y la facilidad de mantenimiento.

Toda esta información debe recopilarse y clasificarse para resolver inconsistencias o conflictos, permitiendo seleccionar un conjunto de requerimientos con coherencia interna.

5.2.3 Trabajar hacia la colaboración: Cuando intervienen múltiples participantes, es común que existan opiniones divergentes sobre los requerimientos del sistema. El rol del ingeniero de requerimientos consiste en fomentar la colaboración identificando las áreas de interés común (puntos de acuerdo) y, fundamentalmente, gestionando las áreas de conflicto o incongruencia para resolver los desacuerdos entre las partes de forma constructiva. La colaboración no exige tomar todas las decisiones por consenso o mediante un comité. Los participantes aportan sus respectivas perspectivas, pero la decisión final sobre qué requerimientos se incluyen recae habitualmente en un "campeón del proyecto" (como un director de negocio o un experto tecnológico).

5.2.4 Hacer las primeras preguntas: En la fase de concepción se plantean preguntas "libres de contexto" para romper el hielo e iniciar la comunicación. Se dividen en tres categorías principales:

- **Centradas en el cliente y beneficios:** Identifican a los participantes, el impacto económico y posibles alternativas existentes (ej. *¿Quién usará la solución?*, *¿Cuál será el beneficio económico?*).
- **Centradas en el problema y la solución:** Permiten entender las percepciones del cliente y el entorno de trabajo (ej. *¿Qué problema resolvería la solución?*, *¿Existen restricciones o requisitos de desempeño?*).
- **Metapreguntas:** Validan la eficacia de la propia entrevista y la precisión de la información recopilada (ej. *¿Sus respuestas son oficiales?*, *¿Debería preguntar algo más?*).

Aunque la entrevista inicial sirve para arrancar el proceso, no es suficiente por sí sola. Para lograr una indagación efectiva, debe ser reemplazada rápidamente por técnicas colaborativas que integren resolución de problemas, negociación y especificación.

5.3 Indagación de Requerimientos: La indagación o recabación combina la solución de problemas, elaboración, negociación y especificación mediante un enfoque colaborativo entre los participantes y el equipo de desarrollo para definir un conjunto preliminar de requerimientos.

5.3.1 Recabación de los Requerimientos en Forma Colaborativa Para llevar a cabo las sesiones colaborativas se aplican las siguientes pautas:

- **Organización:** Se fijan reglas de preparación, una agenda estructurada pero flexible y un facilitador que modera la reunión usando herramientas visuales o digitales.
- **Preparación previa:** A partir de una *solicitud de producto* inicial, cada asistente elabora listas preliminares de objetos, servicios, restricciones y criterios de desempeño.
- **Consolidación de listas:** Durante la reunión se presentan las listas individuales y se crea una lista combinada sin críticas ni debates previos, eliminando redundancias para alcanzar consensos.
- **Miniespecificaciones:** Se redactan explicaciones breves para detallar los objetos o servicios clave de la lista (por ejemplo, dimensiones, teclados e interfaces de pantalla del panel de control). Los puntos complejos que no se resuelvan en el momento se registran en una lista de pendientes para su análisis posterior.

5.3.2 Despliegue de la función de calidad (DFC): Es una técnica de gestión que traduce las necesidades del cliente en requerimientos técnicos, buscando maximizar su satisfacción durante todo el proceso de ingeniería de software. Recopila información mediante entrevistas, encuestas y datos históricos para construir la tabla de la voz del cliente y clasifica los requerimientos en tres categorías:

- **Requerimientos normales:** Objetivos y funciones explícitas solicitadas por el cliente durante las reuniones (ej. tipos de gráficos, funciones específicas y rendimiento deseado). Su presencia genera satisfacción.

- **Requerimientos esperados:** Características implícitas en el sistema que el cliente da por sentadas y no menciona explícitamente (ej. usabilidad, confiabilidad e instalación sencilla). Su ausencia provoca alta insatisfacción.
- **Requerimientos emocionantes:** Funciones innovadoras que van más allá de las expectativas del cliente (ej. capacidades inesperadas o características no solicitadas). Generan un alto impacto positivo si están presentes.

5.3.3 Escenarios de uso: Aunque las reuniones iniciales permiten identificar las funciones del sistema, es indispensable comprender cómo interactuarán los usuarios finales con ellas para avanzar técnicamente. Para lograrlo, los desarrolladores y usuarios construyen escenarios de uso (comúnmente denominados casos de uso), los cuales describen paso a paso cómo se utilizará el sistema en situaciones reales.

5.3.4 Indagación de los productos del trabajo: Los productos resultantes de la actividad de indagación varían según el tamaño del proyecto y deben ser revisados por todos los participantes involucrados. Entre los principales entregables se encuentran:

- **Enunciados de necesidad, factibilidad y alcance:** Delimitan la viabilidad y las fronteras del sistema.
- **Lista de participantes (*stakeholders*):** Registro de clientes, usuarios y colaboradores clave.
- **Descripción del ambiente técnico:** Definición del entorno tecnológico de despliegue.
- **Lista de requerimientos y restricciones:** Organizados por función, incluyendo las reglas de dominio aplicables.
- **Escenarios de uso:** Casos que describen la operación del sistema en distintas condiciones.
- **Prototipos:** Modelos preliminares desarrollados para precisar requerimientos complejos.

5.4 DESARROLLO DE CASOS DE USO: Un caso de uso describe cómo interactúa un usuario final con el sistema en circunstancias específicas bajo la forma de un contrato sobre el comportamiento del software. Se pueden representar mediante texto narrativo, diagramas o formatos estructurados.

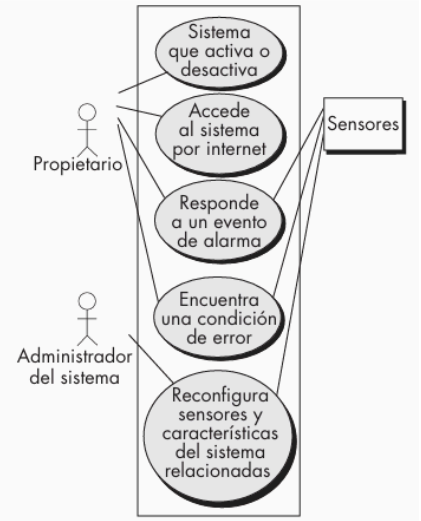
- **Definición de Actores:** El primer paso es identificar los actores, definidos como cualquier entidad externa (personas o dispositivos) que se comunica con el sistema.
    - *Diferencia entre Usuario y Actor:* Un usuario es la persona física, mientras que un actor representa el rol o papel específico que esa persona (o dispositivo) desempeña en un contexto determinado.
    - *Tipos de Actores:* Se clasifican en **principales** (interactúan directamente con el sistema para lograr una función y obtener el beneficio principal) y **secundarios** (proporcionan soporte y servicios para que los principales operen).
- **Preguntas Clave para el Desarrollo:** Para construir un caso de uso, se responden interrogantes sobre quiénes son los actores, sus objetivos, las precondiciones del sistema, las tareas principales, las variaciones, las excepciones y la información de entrada/salida intercambiada.

Caso de uso:	IniciarVigilancia
Actor principal:	Propietario.
Objetivo en contexto:	Preparar el sistema para que vigile los sensores cuando el propietario salga de la casa o permanezca dentro
Precondiciones:	El sistema se ha programado para recibir una clave y reconocer distintos sensores.
Disparador:	El propietario decide “preparar” el sistema, por ejemplo, para que encienda las funciones de alarma
Escenario:	1. Propietario: observa el panel de control 2. Propietario: introduce una clave 3. Propietario: selecciona “permanecer” o “fuera” 4. Propietario: observa una luz roja de alarma que indica que CasaSegura ha sido activada
Excepciones:	1. El panel de control no está listo: el propietario verifica todos los sensores para determinar cuáles están abiertos; los cierra. 2. La clave es incorrecta (el panel de control suena una vez): el propietario introduce la clave correcta. 3. La clave no es reconocida: debe contactarse el subsistema de vigilancia y respuesta para reprogramar la clave. 4. Se elige permanecer: el panel de control suena dos veces y se enciende un letrero luminoso que dice permanecer; se activan los sensores del perímetro. 5. Se selecciona fuera: el panel de control suena tres veces y se enciende un letrero luminoso que dice fuera; se activan todos los sensores.
Prioridad:	Esencial, debe implementarse
Cuándo estará disponible:	En el primer incremento
Frecuencia de uso:	Muchas veces por día
Canal para el actor:	A través de la interfaz del panel de control
Actores secundarios:	Técnico de apoyo, sensores
Canales para los actores secundarios:	Técnico de apoyo: línea telefónica Sensores: interfaces cableadas y frecuencia de radio
Aspectos pendientes:	1. ¿Debe haber una forma de activar el sistema sin usar clave o con una clave abreviada? 2. ¿El panel de control debe mostrar mensajes de texto adicionales? 3. ¿De cuánto tiempo dispone el propietario para introducir la clave a partir del momento en el que se oprime la primera tecla? 4. ¿Hay una forma de desactivar el sistema antes de que se active en realidad?

La revisión sistemática de los casos de uso —tanto de sus versiones básicas como de las detalladas— es fundamental para detectar ambigüedades, inconsistencias o vacíos en la lógica de la interacción antes de avanzar al diseño. Al someter las narrativas y flujos a la validación con los participantes, se garantiza que la visión del comportamiento del sistema sea clara y uniforme para todo el equipo.

FIGURA 5.2

Diagrama de caso de uso de UML para la función de seguridad del hogar de *CasaSegura*



5.5 ELABORACIÓN DEL MODELO DE LOS REQUERIMIENTOS: El objetivo del modelo de análisis es describir los dominios de información, función y comportamiento requeridos para el sistema. Este modelo evoluciona dinámicamente a medida que avanza la comprensión del problema por parte de desarrolladores y participantes, funcionando como una instantánea de los requerimientos en un momento determinado. Durante su desarrollo, los elementos estables sirven de base firme para las etapas de diseño, mientras que los componentes volátiles señalan aspectos que aún requieren mayor aclaración y negociación.

5.5.1 Elementos del modelo de requerimientos: Utilizar múltiples modos de representación para ilustrar el modelo de requerimientos permite analizar el sistema desde distintas perspectivas, aumentando la probabilidad de detectar omisiones, inconsistencias y ambigüedades. Los elementos generales que componen el modelo se dividen en tres categorías principales:

- **Elementos basados en el escenario:** Describen el sistema desde el punto de vista del usuario. Evolucionan desde descripciones textuales y casos de uso básicos hasta diagramas de casos de uso e itinerarios detallados mediante **diagramas de actividades UML**. Sirven como insumo principal para el desarrollo de los demás elementos del modelo.
- **Elementos basados en clases:** Identifican y agrupan los objetos manipulados durante las interacciones en clases que comparten atributos y comportamientos comunes. Se representan mediante **diagramas de clases UML** (que detallan atributos y operaciones) y diagramas de colaboración/relación que muestran la interacción entre dichas clases.
- **Elementos de comportamiento:** Ilustran cómo reacciona el sistema ante eventos externos y cómo cambia su estado operativo. Se representan principalmente mediante **diagramas de estado UML**, los cuales muestran los modos de comportamiento observables, las transiciones impulsadas por eventos y las acciones resultantes.

FIGURA 5.3

Diagramas de actividad del UML para indagar los requerimientos

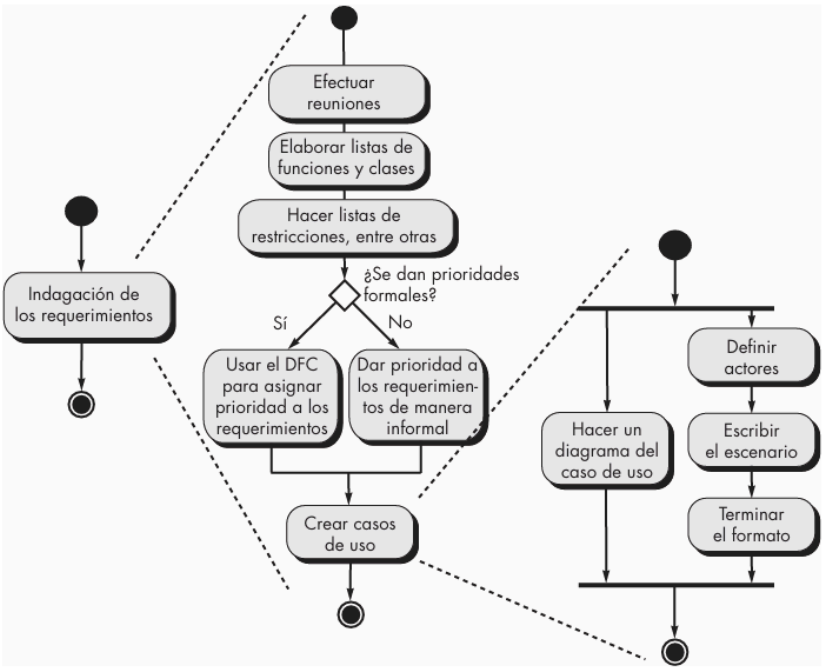


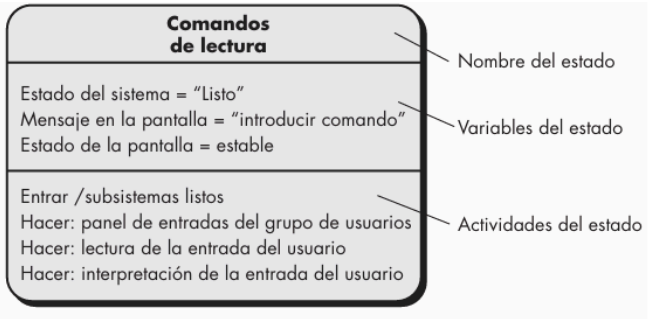
FIGURA 5.4

Diagrama de clase para un sensor



FIGURA 5.5

Notación UML del diagrama de estado



Elementos orientados al flujo:

5.6 REQUERIMIENTOS DE LAS NEGOCIACIONES: Representan cómo la información se transforma a medida que fluye a través del sistema. El sistema acepta entradas (señales de control, datos de teclado, paquetes de red, archivos), aplica funciones de transformación (comparaciones lógicas, algoritmos numéricos, reglas de inferencia) y produce salidas en

diversos formatos (indicadores luminosos, reportes). Este enfoque permite modelar la transformación de datos para cualquier sistema, independientemente de su escala o complejidad.

5.5.2 Patrones de Análisis: Los patrones de análisis son soluciones reutilizables (clases, funciones o comportamientos) a problemas recurrentes dentro de un dominio de aplicación específico. Ofrecen dos beneficios principales:

- **Aceleración del modelado:** Proveen modelos abstractos reutilizables con ventajas y limitaciones documentadas, agilizando la captura de requerimientos.
- **Transición fluida al diseño:** Facilitan la conversión del modelo de análisis en esquemas de diseño al sugerir patrones y soluciones probadas para problemas comunes.

Estos patrones se incorporan al modelo indicando su nombre y se almacenan en repositorios estructurados bajo formatos estándar para permitir su búsqueda y reutilización en futuros proyectos.

5.6 Negociación de Requerimientos y Resultados Ganar-Ganar: La negociación en la ingeniería de requerimientos busca conciliar las diferencias entre las expectativas de los participantes (*stakeholders*) y las limitaciones técnicas o presupuestarias del proyecto:

- **Reconciliación de condiciones:** Consiste en transformar los requerimientos en conflicto o inviables en un conjunto de condiciones de beneficio mutuo (**ganar-ganar**), asegurando que tanto el cliente como el equipo de desarrollo queden satisfechos con el alcance final.
- **Criterio de avance:** Alcanzar este acuerdo negociado es el criterio clave que permite dar por concluida la etapa de negociación e iniciar formalmente las actividades subsiguientes del ciclo de vida del software (especificación, validación y diseño).

5.7 VALIDACIÓN DE LOS REQUERIMIENTOS: A medida que se construyen los elementos del modelo de requerimientos, se examinan formalmente para detectar inconsistencias, omisiones y ambigüedades. Los participantes asignan prioridades a los requerimientos y los agrupan en paquetes para su implementación incremental. La revisión del modelo se realiza mediante un conjunto de preguntas clave agrupadas en criterios fundamentales:

- **Coherencia y Necesidad:** Evalúa si cada requerimiento concuerda con los objetivos generales del sistema, si es realmente indispensable o representa un añadido innecesario, y si se expresa en el nivel adecuado de abstracción.
- **Claridad y Trazabilidad:** Verifica que las especificaciones no sean ambiguas, estén bien delimitadas y cuenten con una fuente clara e individual asignada (*atribución*).
- **Factibilidad y Verificabilidad:** Confirma que no existan requerimientos en conflicto, que sean viables de implementar en el entorno técnico previsto y que puedan someterse a pruebas de validación.

- **Estructura e Integridad del Modelo:** Garantiza que el modelo refleje adecuadamente los dominios de información, función y comportamiento, esté particionado de forma jerárquica para detallar el sistema y utilice patrones de análisis validados y consistentes con las necesidades del cliente.

El cumplimiento de este proceso garantiza que el modelo refleje fielmente las expectativas de los participantes y constituya una base sólida e inalterable para la etapa de diseño.

Bibliografía:

Pressman, R. S. (2010). *Ingeniería del software: Un enfoque práctico* (7.ª ed.). McGraw-Hill.